COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	07/31/26	**DATE SUBMITTED:**	07/31/26
NAME:	Francis Elijah J. Formalejo	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4 **Stacks**

1. PROCEDURE OUTPUT

Mian.cpp:

```cpp
#include<iostream>
#include "stackList.h"

int main(){

    std::cout<< "--- PUSHING 'Y' 'E' 'S' ONTO THE STACK ---- \n";
    //push
    push('S');
    push('E');
    push('Y');

    //display the current top
    Top<char>(); // Top should be 'e'
    displayALL<char>();
    std::cout<<"\n ------ Popping all the elements ----- \n";

    while(!isEmpty<char>()){
        std::cout<<" Popped: "<< pop<char>() <<std::endl;
    }

    std::cout<<"\n ------ Testing the Underflow ----- \n";
    pop<char>();

    return 0;
}
```

StackList.h:

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

//class node that contains data and the next
template <typename T>
class Node{
    public:
    T data;          // VALUE THAT WE NEED STORE
    Node* next;      // POINTER TO THE NEXT NODE
};

//global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

//operations for the stack

// push
template <typename T>
void push(T newData){
    //create a new node and assign the newData
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;

    //new node points to current top, then becomes the new top
    newNode->next = head<T>;
    head<T> = newNode;
}

// pop
template<typename T>
T pop(){
    //check if the stack is empty:
    if(head<T> == nullptr){
        std::cout<< "stack underflow. " <<std::endl;
        return T();
    }

    //save the current top node
    Node<T>* temp = head<T>;
    //store its data
    T tempVal = temp->data;

    // move the head to the next node
```

```
    // move the head to the next node
    head<T> = head<T>->next;
    //free the removed node
    delete temp;
    //return the removed value
    return tempVal;
}

// isEmpty
template<typename T>
bool isEmpty(){
    return head<T> == nullptr;
}

// top
template<typename T>
void Top(){
    // error catching:
    if(isEmpty<T>()){
        std::cout<<"Stack underflow "<<std::endl;
        return;
    }
    // display the Top node
    std::cout<<"Top of Stack: "<<head<T>->data <<std::endl;
}

//display All
template <typename T>
void displayALL(){
    if (head<T> == nullptr){
        std::cout << "stack underflow\n";
        return;
    }

    std::cout<< "Displaying the stack: \n";
    Node<T>* temp = head<T>;
    while(temp != nullptr){
        std::cout << temp->data << "\n";
        temp = temp->next;
    }
    std::cout << "\n";
```

```
}

#endif
```

Output:

```
--- PUSHING 'Y' 'E' 'S' ONTO THE STACK ----
Top of Stack: Y
Displaying the stack:
--- PUSHING 'Y' 'E' 'S' ONTO THE STACK ----
Top of Stack: Y
Displaying the stack:
Y
Top of Stack: Y
Displaying the stack:
Y
E
Displaying the stack:
Y
E
Y
E
S
E
S
S



------ Popping all the elements -----
Popped: Y
Popped: E
Popped: S

------ Testing the Underflow -----
stack underflow.
```

Push was used to add new stuff ont the top of the stack
Pop is doing the removal of items in the stack
Isempty is the one who checks items in the stack if its empty or not
Displayall print the items in the stack from top to bottom

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Main.cpp:

```cpp
#include <iostream>
#include <string>
#include "stackList.h"

bool isMatch(char open, char close){
    if(open == '(' && close == ')') return true;
    if(open == '{' && close == '}') return true;
    if(open == '[' && close == ']') return true;
    return false;
}

bool checkBalance(std::string expr){
    while(!isEmpty<char>()){
        pop<char>();
    }

    for(int i = 0; i < expr.length(); i++){
        char ch = expr[i];
        if(ch == '(' || ch == '{' || ch == '['){
            push(ch);
        }
        else if(ch == ')' || ch == '}' || ch == ']'){
            if(isEmpty<char>()){
                return false;
            }
            char topChar = pop<char>();
            if(!isMatch(topChar, ch)){
                return false;
            }
        }
    }
    return isEmpty<char>();
}

int main(){
    std::string tests[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B]+[C-D]}"
    };
```

```
    for(int i = 0; i < 4; i++){
        std::cout << tests[i] << std::endl;
        if(checkBalance(tests[i])){
            std::cout << "Y" << std::endl;
        }else{
            std::cout << "N" << std::endl;
        }
    }

    return 0;
}
```

StackList.h:

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

#include <iostream>

template <typename T>
class Node{
    public:
    T data;
    Node* next;
};

template <typename T>
Node<T> *head = nullptr;

template <typename T>
void push(T newData){
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = head<T>;
    head<T> = newNode;
}

template<typename T>
T pop(){
    if(head<T> == nullptr){
        return T();
    }
    Node<T>* temp = head<T>;
    T tempVal = temp->data;
    head<T> = head<T>->next;
    delete temp;
    return tempVal;
}

template<typename T>
bool isEmpty(){
    return head<T> == nullptr;
}

template<typename T>
void Top(){
    if(isEmpty<T>()){
        std::cout<<"Stack underflow "<<std::endl;
        return;
    }
    std::cout<<"Top of Stack: "<<head<T>->data <<std::endl;
}
```

```
#endif
```

Output:

```
(A+B)+(C-D)
Y
((A+B)+(C-D)
N
((A+B)+[C-D])
Y
((A+B]+[C-D]}
N
```

Expression	Valid (Y/N)	Output (Screenshot)	Analysis
(A+B)+(C-D)	Y	(A+B)+(C-D) Y ((A+B)+(C-D) N ((A+B)+[C-D]) Y ((A+B]+[C-D]} N	Valid because it has even and alike opening and closing
((A+B)+(C-D)	N	(A+B)+(C-D) Y ((A+B)+(C-D) N ((A+B)+[C-D]) Y ((A+B]+[C-D]} N	Lacking 1 close parenthesis
((A+B)+[C-D])	Y	(A+B)+(C-D) Y ((A+B)+(C-D) N ((A+B)+[C-D]) Y ((A+B]+[C-D]} N	Parenthesis matches each closing ones

((A+B]+[C-D]}	**N**	(A+B)+(C-D) Y ((A+B)+(C-D) N ((A+B)+[C-D]) Y ((A+B]+[C-D]} N	**The opening and closing are not alike**

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

Conclusion:

Overall, this activity was slight difficult for me because our professor wasnt around to explain some headstart but its fine since I caught up to the pace eventually. In this I learned how stacks work by using linked lists and pointers to store items in a first and last order. The main code shows how functions like push, pop, isempty, remove, and check items without errors. Lastly, I think I did mid and I'll rate it a 7/10 difficulty task, and area of improvement is still my code memorization (stock knowledge) and faster coding.